Лабораторна робота 1. Шифри заміни та перестановки

Програмні технології захисту інформації

Виконав(-ла): [ПІБ], група [ГРУПА]

Перевірив(-ла): [ПІБ викладача]

Дата: [ДАТА]

1. Тема, мета, інструменти

Тема: шифри заміни та перестановки.

Мета: навчитися формально задавати ключі, реалізовувати взаємно однозначні перетворення, виконувати зашифрування і розшифрування та перевіряти програму тестами.

Інструменти:

Інструмент	Версія / призначення
Python (CPython)	3.15.0 і 3.9.6, лише стандартна бібліотека
`unicodedata`	нормалізація NFC, категорії символів
`argparse`	консольний інтерфейс
`unittest`	автоматизовані тести
Git	історія змін
ОС	macOS 15 (Darwin 24.4.0, arm64)
Генеративний ШІ	Claude Code (Anthropic), див. розділ 12

2. Теоретичні відомості

Підстановка змінює символи, але не їхні позиції. Ключем простої підстановки є перестановка B алфавіту A: літера `A[i]` замінюється на `B[i]`. Перетворення оборотне тоді й лише тоді, коли B містить кожну з 33 літер рівно один раз (бієкція A → B). Розшифрування використовує обернене відображення `B[i] → A[i]`. Оскільки кожна літера завжди замінюється тією самою, частоти літер, довжини слів і шаблони повторів зберігаються, тож довгий шифротекст піддається частотному аналізу.

Таблицю B будують із ключового слова W: нормалізують регістр, вилучають повторні літери (залишаючи першу появу) і дописують решту літер A в алфавітному порядку.

Шифр Віженера є періодичною багатоалфавітною підстановкою. Літери та ключ перетворюють на індекси 0..32, і рахує лише літери алфавіту:

```
C_i = (P_i + K_(i mod l)) mod 33,     P_i = (C_i − K_(i mod l)) mod 33.
```

Одна літера відкритого тексту може переходити в різні літери шифротексту, що вирівнює розподіл частот. Проте ключ повторюється з періодом l. Підпослідовності з кроком l зашифровані простим зсувом, тому довжину ключа можна знайти методом Касіскі або за індексом збігів, а потім розв’язати l простих зсувів.

Перестановка не змінює набір символів, лише їхні позиції. У **блоковій** перестановці для кожного повного блока x довжини n формується `y_j = x_(π_j)`. Обернена перестановка задовольняє `π⁻¹[π_j] = j`; для $\pi = (3, 2, 5, 1, 4)$ маємо $\pi^{-1} = (4, 2, 1, 5, 3)$. У **вертикальній**

перестановці текст записують по рядках у прямокутник із k стовпцями і читають стовпці в порядку ключа. Частоти літер при перестановці не змінюються взагалі.

3. Специфікація обробки тексту

Аспект	Прийняте правило
Алфавіт	`АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ`, m = 33, індекси 0..32
Кодування	вхідні й вихідні файли UTF-8; BOM на вході ігнорується; `\n` і `\r\n` не змінюються
Нормалізація	NFC → `upper()` → NFC (друга NFC — бо `upper()` може дати розкладену форму)
Регістр	базовий режим: усі літери у верхньому регістрі; збереження регістру не реалізоване
Допустимі символи	33 літери, пробіли, табуляція, переноси рядків, цифри (Nd), розділові знаки (P*), апострофи `' ’ ʼ`, `№`, математичні (Sm) та валютні (Sc) знаки
Недопустимі символи	латиниця, інші кириличні літери (Ы, Ё, Ъ, Э), емодзі, керувальні й окремі комбіновані знаки. Результат — повідомлення з кодом символу та позицією, без аварійного завершення
Підстановки	символи поза алфавітом зберігаються на своїх місцях і **не** просувають позицію ключа Віженера
Перестановки: `preserve` (типовий)	переставляється весь підготовлений текст, включно з пробілами та переносами; повністю зворотний
Перестановки: `letters`	спершу вилучаються всі символи, крім літер; незворотний щодо форматування, еталон — підготовлений рядок без пробілів
Неповний блок (блокова)	останній блок довжини < n залишається без змін
Неповний рядок (вертикальна)	доповнюється службовою літерою (типово `Х`), кількість p ($0 \le p < k$) повідомляється користувачу й передається при розшифруванні; вилучаються рівно p останніх символів

Еталон зворотності: `D_k(E_k(x)) = prepare(x)`, де `prepare` — нормалізація і зміна регістру.

4. Перевірка ключів

Ключ перевіряється до читання й обробки тексту. Повідомлення пояснює, що саме не так.

Алгоритм	Умова прийняття	Приклад відмови
Проста підстановка	ключове слово не порожнє, лише літери А; таблиця має довжину 33, лише літери А, без повторів	`таблиця не є перестановкою: повторюються А, відсутні Я.`
Віженер	після нормалізації не порожній і містить лише літери А	`ключ Віженера містить символ 'К' поза алфавітом …`

Алгоритм	Умова прийняття	Приклад відмови
Блокова / вертикальна	числа, розділені пробілами чи комами; кожне від 1 до n рівно один раз	`ключ не є перестановкою: повторюються [2], відсутні [3].` / `число 4 поза діапазоном 1..3`
Доповнення	ціле, $0 \le p < k$; довжина шифротексту кратна k	`кількість доданих символів 4 має бути в межах 0..3`

5. Структура програми та ключові рішення

```
classic_ciphers/
  text.py          алфавіт, нормалізація, перевірка символів, режими перестановок
  substitution.py  build_table, validate_table, encrypt, decrypt
  vigenere.py      parse_key, encrypt, decrypt
  permutation.py   parse_permutation, validate_permutation, invert
  block.py         encrypt, decrypt (неповний блок без змін)
  columnar.py      encrypt -> (text, padding), decrypt(text, key, padding)
  analysis.py      частоти, індекс збігів, оцінка довжини ключа (додатково)
  fileio.py        read_text, write_text_atomic
  errors.py        CipherError та підкласи
  cli.py           команди encrypt, decrypt, rules, analyze
tests/             88 тестів unittest
experiments/       run_experiment.py та results/
```

Ключові рішення:

1. **Криптографія відокремлена від інтерфейсу.** Модулі алгоритмів не читають і не друкують нічого, лише приймають рядки й повертають рядки або кидають `CipherError`. Тому всі алгоритми тестуються без ручного введення, а `cli.py` лише розбирає аргументи і перетворює помилки на повідомлення `Помилка: …` та код завершення 1.

2. **Одна функція підготовки тексту** (`text.prepare`) для всіх алгоритмів, тож правила нормалізації неможливо застосувати по-різному в різних шифрах.

3. **Розшифрування перестановок повторно використовує зашифрування.** Для блокової перестановки обернена перестановка обчислюється один раз (`invert`), а потім виконується той самий прохід `_apply`, як радить псевдокод методички:

```python
def _apply(text, perm):
    n = len(perm)
    blocks = []
    for start in range(0, len(text), n):
        block = text[start:start + n]
        if len(block) < n:
            blocks.append(block)            # політика неповного блока
        else:
            blocks.append("".join(block[source - 1] for source in perm))
    return "".join(blocks)
```

4. **Вертикальна перестановка повертає кількість доповнення окремо** (`Encrypted(text, padding)`), а не вгадує її за кінцем рядка. Тому текст, який сам закінчується на `X`, відновлюється правильно (окремий тест `test_only_known_number_of_pad_chars_removed`). Стовпець с прямокутника береться зрізом `grid[c-1::k]`.

5. **Віженер** просуває лічильник ключа лише на літерах алфавіту. Від'ємна різниця при розшифруванні приводиться до 0..32 операцією `%`, яка в Python для додатного модуля завжди повертає невід'ємний результат.

6. **Атомарний запис файлів.** Результат пишеться у тимчасовий файл у тому ж каталозі, а потім `os.replace` підміняє ціль. Якщо ключ чи текст некоректні, вихідний файл не створюється. Якщо запис не вдався, наявний файл не пошкоджується.

7. **Файли читаються з `newline=""`**, щоб переноси `\r\n` не перетворювалися і режим `preserve` був точно зворотним для файлів.

6. Контрольні вектори

Отримано командою `python3 experiments/run_experiment.py` (файл `experiments/results/summary.md`).

Алгоритм	Ключ	Відкритий текст	Очікуваний	Фактичний	Розшифровано	Статус
Зворотний алфавіт	В = А у зворотному порядку	СЛАВА УКРАЇНІ	ІНЯЬЯ ЗОЇЯРЛС	ІНЯЬЯ ЗОЇЯРЛС	СЛАВА УКРАЇНІ	OK
Віженер	КЛЮЧ	СЛАВА УКРАЇНІ	ВЬЮЩК ДЇККЧЛД	ВЬЮЩК ДЇККЧЛД	СЛАВА УКРАЇНІ	OK
Блокова перестановка (letters)	3 2 5 1 4	СЛАВА УКРАЇНІ	АЛАСВРКЇУАНІ	АЛАСВРКЇУАНІ	СЛАВАУКРАЇНІ	OK
Вертикальна перестановка	3 1 4 2	КРИПТОГРАФІЯ	ИГІКТАПРЯРОФ	ИГІКТАПРЯРОФ	КРИПТОГРАФІЯ	OK

Перевірка вручну для Віженера: С(21) + К(14) = 35 mod 33 = 2 → В; Л(15) + Л(15) = 30 → Ь; А(0) + Ю(31) = 31 → Ю; В(2) + Ч(27) = 29 → Щ; пробіл пропускається, ключ не просувається; А(0) + К(14) = 14 → К. Отже, ключ просувається лише на літерах.

7. Тестування

Середовище: CPython 3.15.0 і 3.9.6, macOS 15 arm64. **Команда:** `python3 -m unittest -v`. **Підсумок:** `Ran 88 tests … OK` на обох версіях Python, тобто 88 тестів, 0 невдач, 0 помилок.

Модуль тестів	Група з методички	Кількість
`test_vectors.py`	Контрольні	6

Модуль тестів	Група з методички	Кількість
`test_roundtrip.py`	Зворотність: 12 фіксованих повідомлень × 4 алгоритми (× 2 режими для перестановок) та генеративні тести, 4 × 300 випадкових ключів і текстів із фіксованим зерном 20261012	10
`test_text.py`	Українські літери, NFC, форматування, непідтримувані символи	12
`test_boundaries.py`	Границі, доповнення	10
`test_keys.py`	Негативні (ключі, доповнення)	23
`test_files.py`	Файли UTF-8	8
`test_cli.py`	Інтерфейс, коди завершення, файли через CLI	12
`test_analysis.py`	Додаткові завдання	7
Разом		**88**

Вибрані тести з вхідними даними та результатами:

№	Група	Вхідні дані	Очікуваний результат	Фактичний	Статус
1	Зворотність	Віженер, ключ ҐЄЄІЇ, `Їжачок і в яблука, а ґава — горіхи!`	`ЇЖАЧОК ЇВ ЯБЛУКА, А ҐАВА — ГОРІХИ!`	збігається	OK
2	Зворотність	4 × 300 випадкових текстів (0–60 символів) і ключів	D(E(x)) = prepare(x)	для всіх 1200	OK
3	Укр. літери	підстановка, B = зворотний алфавіт, `ґєії`	`ШХСР`	`ШХСР`	OK
4	NFC	`И` + U+0306, `І` + U+0308 (розкладені)	те саме, що для `Й Ї`	збігається	OK
5	Границі	порожній текст, усі 4 алгоритми	`""`, доповнення 0	`""`, 0	OK
6	Границі	Віженер, `Я`, ключ `Б` / розшифрування `А`	`А` / `Я`	`А` / `Я`	OK
7	Границі	блокова 3 2 5 1 4, довжини 4 / 5 / 6	`АБВГ` / `ВБҐАГ` / `БҐАГД`	збігається	OK
8	Границі	вертикальна 3 2 5 1 4, довжини 4 / 5 / 6	доповнення 1 / 0 / 4, відновлення	збігається	OK

№	Група	Вхідні дані	Очікуваний результат	Фактичний	Статус
9	Доповнення	вертикальна 3 1 2, `ПОРОХ ХХ` (закінчується на службову літеру)	`ПОРОХ ХХ` після розшифрування	збігається	OK
10	Форматування	Віженер `А, 1\n А`, ключ `БВ`	`Б, 1\nВ` (ключ не просувається)	збігається	OK
11	Форматування	блокова, `--mode letters`, `А Б, В!`	`АБВ`	`АБВ`	OK
12	Негативні	Віженер, ключ `""` / `"KEY"`	помилка «порожній» / «поза алфавітом»	помилка	OK
13	Негативні	перестановка `1 2 2`	«повторюються [2], відсутні [3]»	збігається	OK
14	Негативні	перестановка `0 1 2`, `1 2 4`, `325 14`	«поза діапазоном»	помилка	OK
15	Негативні	таблиця з 33 літер, де `Я` замінено на `А`	«повторюються А, відсутні Я»	збігається	OK
16	Негативні	вертикальна, `--pad 4` при k = 4	помилка діапазону 0..3	помилка	OK
17	Негативні	текст `Hello`	помилка з `U+0048`, код завершення 1, без трасування	збігається	OK
18	Файли	багаторядковий UTF-8 файл із `\r\n` і завершальним `\n`, усі 4 алгоритми через CLI	вміст відновлено байт у байт (у верхньому регістрі)	збігається	OK
19	Файли	файл у cp1251	помилка «не є коректним UTF-8»	збігається	OK
20	Файли	збій `os.replace` під час запису	старий вміст файлу не змінено, тимчасових файлів немає	збігається	OK

8. Власний текст: ключі, шифротексти, розшифрування

Відкритий текст: `samples/message.txt`. Він складається з 570 символів, із них 482 літери алфавіту. Текст містить Ґ, Є, І, Ї, апострофи, розділові знаки та переноси рядків.

```
Історія криптографії починається задовго до появи комп'ютерів. Ще в античні часи полков
одці ховали зміст наказів, переставляючи літери або замінюючи їх іншими знаками. Спарта
нці намотували стрічку на скиталу певної товщини.
У шістнадцятому столітті Блез де Віженер описав багатоалфавітний шифр, який довго вважа
ли незламним. Проте у дев'ятнадцятому столітті Фрідріх Касіскі показав, що повтори у ши
фротексті видають довжину ключа.
Сьогодні такі шифри вивчають лише як навчальні приклади: вони пояснюють ґрунтовні ідеї
підстановки й перестановки, але не захищають даних.
```

Алгоритм	Ключ	Режим	D(E(x)) = prepare(x)
Проста підстановка	ключове слово КРИПТОГРАФІЯ → В = К РИПТОГАФІЯБВҐДЕЄЖЗЇЙЛ МНСУХЦЧШЩЬЮ	—	OK
Віженер	ЗАХИСТ	—	OK
Блокова перестановка	3 2 5 1 4	preserve	OK
Вертикальна перестановка	4 1 3 7 2 6 5 (додано 4 службові літери)	preserve	OK

Проста підстановка:

```
БЛМЗЙБЮ ДЙЯЇМЗПЙКСБВ ЇЗЦЯЖКАМЩЛЮ ІКОЗИПЗ ОЗ ЇЗЮИЯ ДЗЄЇ'ЬМГЙБИ. ШГ И КЖМЯЦЖБ ЦКЛЯ ЇЗЕДЗИ
ЗОХБ УЗИКЕЯ ІЄБЛМ ЖКДКІБИ, ЇГЙГЛМКИЕЮЬЦЯ ЕБМГЙЯ КРЗ ІКЄБЖЬЬЦЯ ВУ БЖЧЯЄЯ ІЖКДКЄЯ. ЛЇКЙМК
ЖХБ ЖКЄЗМНИКЕЯ ЛМЙБЦДН ЖК ЛДЯМКЕН ЇГИЖЗВ МЗИШЯЖЯ.
Н ЧБЛМЖКОХЮМЗЄН ЛМЗЕБММБ РЕГІ ОГ ИБФГЖГЙ ЗЇЯЛКИ РКПКМЗКЕСКИБМЖЯҐ ЧЯСЙ, ЮДЯҐ ОЗИПЗ ИИКФК
ЕЯ ЖГІЕКЄЖЯЄ. ЇЙЗМГ Н ОГИ'ЮМЖКОХЮМЗЄН ЛМЗЕБММБ СЙБОЙБУ ДКЛБЛДБ ЇЗДКІКИ, ШЗ ЇЗИМЗЙЯ Н ЧЯ
СЙЗМГДЛМБ ИЯОКЬМЩ ОЗИФЯЖН ДЕЬЦК.
ЛЩЗПЗОЖБ МКДБ ЧЯСЙЯ ИЯИЦКЬМЩ ЕЯЧГ ЮД ЖКИЦКЕЩЖБ ЇЙЯДЕКОЯ: ИЗЖЯ ЇЗЮЛЖЬЬМЩ ТЙНЖМЗИЖБ БОГВ
ЇБОЛМКЖЗИДЯ Ґ ЇГЙГЛМКЖЗИДЯ, КЕГ ЖГ ІКУЯШКЬМЩ ОКЖЯУ.
```

Віженер:

```
РСКШЖАЖ КЇРЄІЧГЇИЇАС ПИҐЮЕЗЄКЄЗС ОАЬШУХЧ ДИ ЩЕСІИ ЕШҐЖ'ЄТЮЬЯФ. ДЕ Ч ИДІПЧЗС ЛТЬИ ІШГГЧВ
ИЛКА БОЧИГЯ ОМГЮИ ЕЗКХПЯФ, ШЕЇМЗІЗВЄЗПМП ЛГЯЧЗП АЦШ ЬТХІЗЖПМП ЇН СДНПМВ ПДТУАЖР. ЗЖЗРКИ
ДЛР НХЦЕІЯВХХЮ ИЮРГҐВЇ ЦА ЙФЮІЗЛЛ ЩЧФЦОҐ ЯЕФДИЗР.
І НРСКЧСЧВЯКШҐЇ ЬТИХЯІЮІ ЦХЧЮ КЕ ЧСЩШЦЕЇ ШЄЯЬАЧ ІСХЗТИИГЙЗВГЯДЯТ ШВБЖ, СУИД ЛЕФЇО ЧЇСЬЗ
ЛВ ЧЧЮФАЖЧЮД. ШРИЯЧ Ї КЕЧ'ЗИЕЗДОЗИЄХУ ЙЯЕҐРТКС ЇЗРДЇСЙ ГЗСГЮВА ШОЕИЬТІ, ЩИ ЩЕФЮОЇР І НП
ФЇШИШУСКС УЯКАУЯО ЧЧВАРДЇ УЛУҐС.
ИЕОШШЦЕР ТХФЯ НПФЇР УЯІЧХЖИП ФИРМ РГ ЦАЧҐСҐЕНГ ЩЖЯУЛХЛЮ: ФЧНВ ЩЕСЬНУЖИП ЙРЛЧИЄІНГ СЦШС
ПГЛЗІЗНИЇВЯ Т ПЮЬЧИЮАЗШУГП, АЄМ ДШ ОАНРНТЄТТ ЛСЕПХ.
```

Блокова перестановка (у режимі preserve переносяться також пробіли й переноси рядків):

```
ТСРІО ЯРІКТПГИОФАЇРІОПИ ЧЄАЬНТ ЯАСЗВООДГОДП  ВЯ ОИМО'КПЕТІЮР .ЕВЩ ВН АЧИІТНАЧИ СОПК ЛОВ
ЦОДХ ВІОИЛЗА СІ МТКАЗНА,ВПІ ЕРТЕСЛВЮАЯ ИІЧЛРЕ ТИОБЗА ІМЮАНИЧЇЮ І ШХНИМЗИ КАМНА .ПИСТРНА
А ІАЦНТОВМУИЛСА ІРКТЧН  УАИКАСТ УЕЛПОН ВЇВОИТЩ.ИУН
ІШТ СДАЯНЦМО ТУОТІСЛІТБТ ЗЕДЛ В ЖЕІЕН ЕРИПАОСБ ГВАОТЛААВАТФІЙИШН РФ И,ИК ЯЙВООДГВВЖ АИЛ
НА ЛЗМЕАМИ Н.ОРЕПТ УЕ ДЯ'НВТЦДТАЯУМСО ЛОТТІ ІРТФРДХІІАКІ СІКПС АКАОЗ ,ОВЩОПТ ВИРУО ИШР
ФЕТСОК ІИТВЮАЬДТОДЖ ВУНКИ ЧЮ.ЛАЬСГ
ОНД ОІКА ТІФИИШРИВЧ ВТЮ АЬШИ ЛЕ КАЯНАЧЬВЛ ІРНПЛКДИА :ОИВ ИОНПНСЮЯЮ ЬРТҐТНВУО ІДНІ ЇІЕПТ
```

```
СНДАКВ ОИП РЙЕТСНЕАКВ,ОИЛА  Е ЕАНЗЩИЮХА ЬАТДХИ
Н.
```

Вертикальна перестановка:

```
ОИФИЯГОМІВН ОВІАЕЛІБНХИМРНАІ УЇНІЯТ ЕЕАТІИИОЛАП НМІІАП,В ЕИД
Н ВЬЯАР:ПТТДСИЕК ИДХІ ГПТДО ТЩТАК  Н,ТИИАИШАСЦТС ТВВУАУТЗЖПАФЙ ОАЕМЕ'ЯТ ХКЗ ИРІТИЧГАРАШ
АІАННРІПОПНАЗТХТРАЧСВПОР ЧИВОМКПВЛАІЇМАА ВРАЛОИШЦСІДНСАВШКГАЛ УТОЛРК ВОУТВ У.ДІ Т ЧПИ Ю
НІДКРВЕХ
ЯО ЄАДИЮ НЧЛІИ ВСЧРЗЧНН НО УИЕО
НМІЕІОБЛИ,ДВНИТВЦСІІСАОРФТЮЖЮОТФЧИННЛОСҐН Н А  ЮИХСКРОЬО КЕЕИСОХЗА А  М ИКПІУТНАНЩ Д Т
ЕИГА ЯВЖЗ. ЯТОФ ІАП О ЬНАОКИЮЕВ ДИЮУ ІВЕОЛАЬ.ІТЇАЗ В'.А ОЦЛТІЕЮЕ ЮІЗ.АМИККПТ.ТОЛЛВ  АНР
В НОЕД ТРІКЩОИСАВЛЬ ИВЛ ЬКВЯ ВЇАЙТ,ЕАНХРПІН ОЯПВ ІПДАСЗРЯТОЮ  ИТАЛЧС  ИСТОБ РВОТФЙ ИМРД
АУТДСО ТШКДОКСІШИ КЛИ ОЬОЕТ СИНЩАХ
```

Результат розшифрування для всіх чотирьох алгоритмів однаковий: це підготовлений текст.

```
ІСТОРІЯ КРИПТОГРАФІЇ ПОЧИНАЄТЬСЯ ЗАДОВГО ДО ПОЯВИ КОМП'ЮТЕРІВ. ЩЕ В АНТИЧНІ ЧАСИ ПОЛКОВ
ОДЦІ ХОВАЛИ ЗМІСТ НАКАЗІВ, ПЕРЕСТАВЛЯЮЧИ ЛІТЕРИ АБО ЗАМІНЮЮЧИ ЇХ ІНШИМИ ЗНАКАМИ. СПАРТА
НЦІ НАМОТУВАЛИ СТРІЧКУ НА СКИТАЛУ ПЕВНОЇ ТОВЩИНИ.
У ШІСТНАДЦЯТОМУ СТОЛІТТІ БЛЕЗ ДЕ ВІЖЕНЕР ОПИСАВ БАГАТОАЛФАВІТНИЙ ШИФР, ЯКИЙ ДОВГО ВВАЖА
ЛИ НЕЗЛАМНИМ. ПРОТЕ У ДЕВ'ЯТНАДЦЯТОМУ СТОЛІТТІ ФРІДРІХ КАСІСКІ ПОКАЗАВ, ЩО ПОВТОРИ У ШИ
ФРОТЕКСТІ ВИДАЮТЬ ДОВЖИНУ КЛЮЧА.
СЬОГОДНІ ТАКІ ШИФРИ ВИВЧАЮТЬ ЛИШЕ ЯК НАВЧАЛЬНІ ПРИКЛАДИ: ВОНИ ПОЯСНЮЮТЬ ҐРУНТОВНІ ІДЕЇ
ПІДСТАНОВКИ Й ПЕРЕСТАНОВКИ, АЛЕ НЕ ЗАХИЩАЮТЬ ДАНИХ.
```

Усі файли збережено в `experiments/results/`. Повторний запуск `python3 experiments/run_experiment.py` дає ідентичні результати, бо в програмі немає випадковості, а SHA-256 вхідного файлу зафіксовано в `summary.md`.

9. Додаткові завдання: частотний аналіз

Команда `python3 -m classic_ciphers analyze` будує гістограму частот, обчислює індекс збігів (IC) і оцінює довжину ключа Віженера.

Ранг	Відкритий текст	Шифротекст простої підстановки
1	А — 45	К — 45
2	О — 38	З — 38
3	И — 37	М — 37
4	Т — 37	Я — 37
5	І — 31	Б — 31
6	Н — 30	Ж — 30
7	В — 29	И — 29
8	Е — 20	Г — 20
9	Р — 19	Й — 19
10	С — 19	Л — 19

Розподіл частот простої підстановки є точною перестановкою розподілу відкритого тексту. IC однаковий: 0,0495 і 0,0495. Тому найчастіша літера шифротексту К майже напевно відповідає А, і далі ключ відновлюється за частотами. У Віженера IC = 0,0332, що близько до випадкового тексту (1/33 = 0,0303), тобто розподіл вирівнюється. Проте середній IC стовпців із кроком L різко зростає саме при L = 6 (0,0485) та L = 12 (0,0481), тож метод індексу збігів правильно визначив довжину ключа ЗАХИСТ = 6. Тест `test_estimates_key_length_on_sample` перевіряє це також для ключів довжини 4 і 5.

10. Скриншоти

Скриншоти отримано під час реального запуску команд у каталозі проєкту.

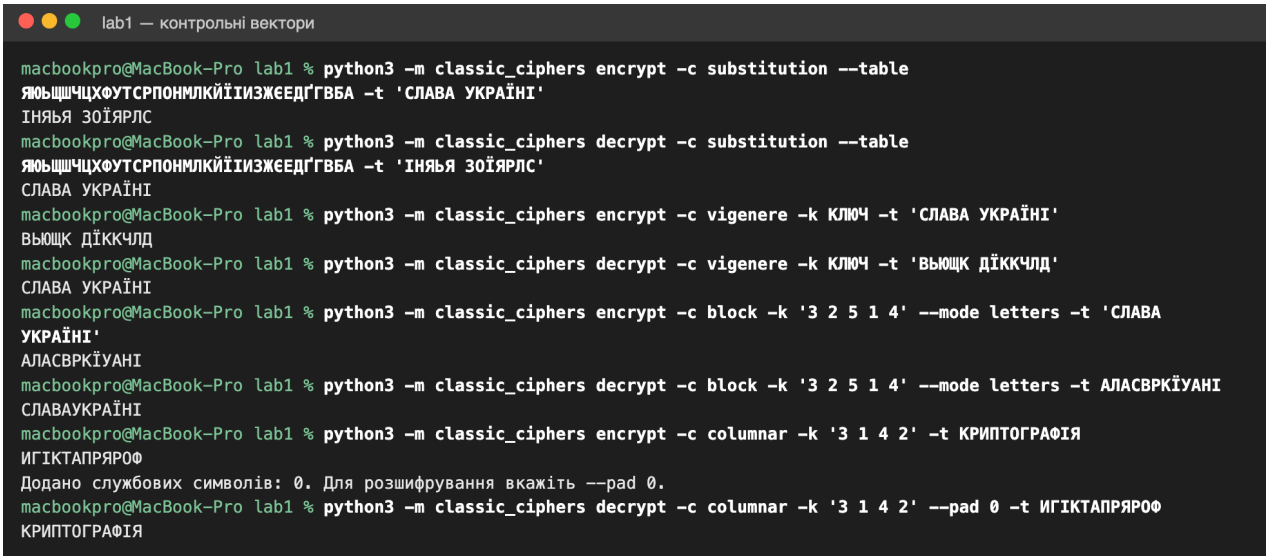


Рисунок 1 — Виконання контрольних векторів у терміналі: `encrypt` / `decrypt` для чотирьох алгоритмів

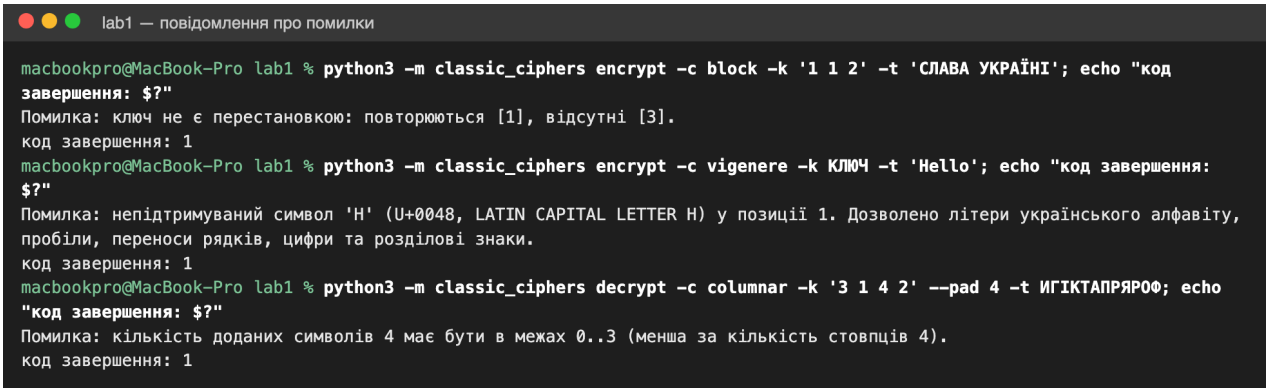


Рисунок 2 — Повідомлення про помилки: повтор у перестановці (`-k "1 1 2"`), непідтримуваний символ і завелика кількість доповнення; код завершення 1

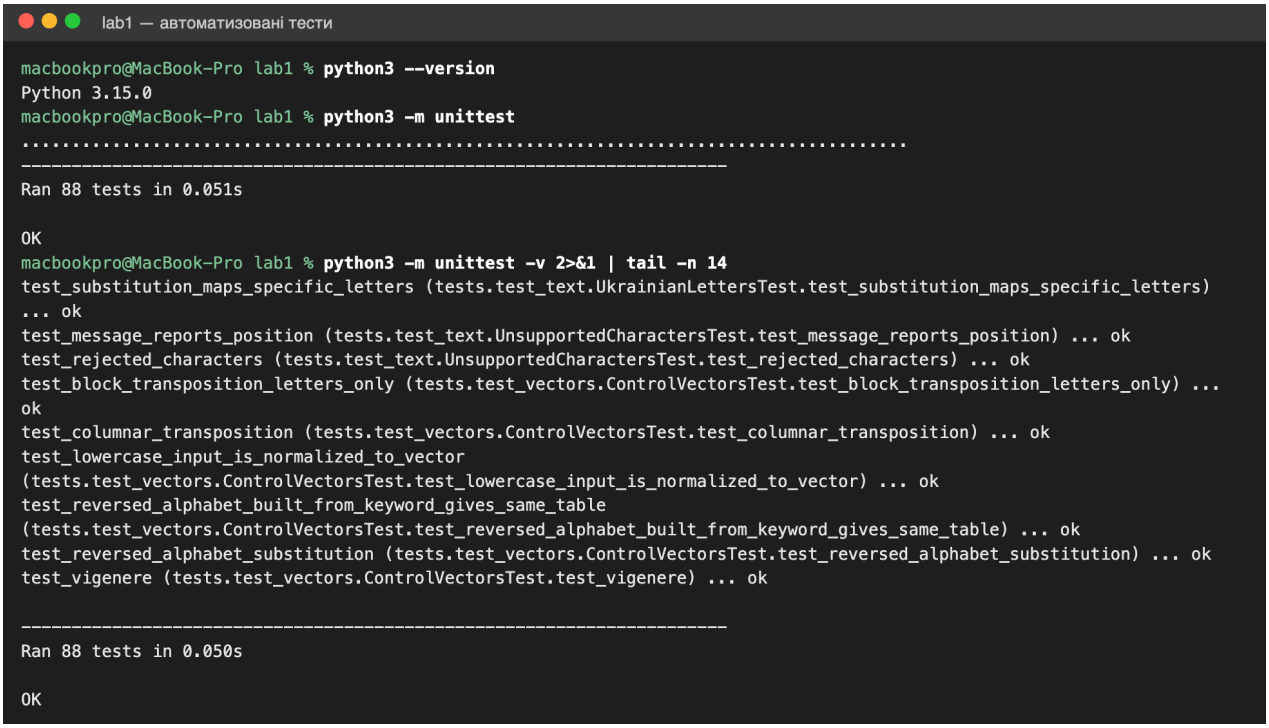


Рисунок 3 — Запуск тестів: `python3 -m unittest` (88 тестів, OK) і фрагмент докладного виводу `-v`

11. Висновки

1. Реалізовано чотири обов’язкові алгоритми з окремими функціями зашифрування, розшифрування й перевірки ключів. Усі контрольні вектори збігаються з очікуваними, а зворотність підтверджено на 12 фіксованих повідомленнях, 1200 генеративних прикладах і власному тексті з 482 літер.
2. Головні джерела помилок реалізації, які довелося явно специфікувати й покрити тестами:
    - порядок алфавіту, зокрема місця Ґ, Є, І, Ї;
    - просування ключа Віженера на пробілах;
    - від’ємна різниця при розшифруванні;
    - змішування 0- та 1-індексації у перестановках;
    - політика неповного блока;
    - видалення «всіх X наприкінці» замість відомої кількості доповнення;
    - розкладені Unicode-послідовності (Й як И + U+0306);
    - зміна переносів рядків під час читання файлу.
3. Властивість `D(E(x)) = x` гарантує лише коректність, а не безпеку. Проста підстановка зберігає частоти літер (ІС шифротексту дорівнює ІС тексту). Віженер приховує частоти лише частково: період ключа виявляється індексом збігів навіть на тексті з менш ніж 500 літер. Перестановки зберігають склад літер повністю. Жоден із шифрів не забезпечує цілісності чи автентичності: зміну шифротексту неможливо виявити. Для реальних даних слід використовувати стандартизоване автентифіковане шифрування (наприклад, AES-GCM, FIPS 197, NIST SP 800-38D) з перевірених бібліотек.
4. Обмеження реалізації:

    - регістр не зберігається;
    - символи поза задокументованою множиною (латиниця, емодзі) відхиляються;
    - графемні кластери не обробляються;
    - під час виведення на екран додається завершальний перенос рядка, тому точну зворотність форматування гарантує робота з файлами.

12. Використання генеративного ШІ

Під час виконання роботи використано **Claude Code (Anthropic)** для:

- генерації коду програми й тестів;
- складання README;
- підготовки чернетки цього звіту.

Самостійно перевірено: [заповніть, наприклад: ручний розрахунок контрольних векторів, запуск тестів, розбір кожного модуля, відповіді на контрольні запитання].

13. Посилання

- Репозиторій із кодом, тестами й README: https://github.com/brahayarn/lab1-classic-ciphers
- NIST FIPS 197, Advanced Encryption Standard (AES), updated 2023
- NIST SP 800-38D, Galois/Counter Mode (GCM) and GMAC
- OWASP Cryptographic Storage Cheat Sheet
- Unicode Standard Annex #15, Unicode Normalization Forms

Додаток. Відповіді на контрольні запитання

1. **Що зберігається і що змінюється?** Підстановка змінює символи і зберігає позиції, тобто структуру тексту. Перестановка змінює позиції і зберігає набір символів та їхні частоти.
2. **Чому одноалфавітна підстановка зберігає частотний розподіл?** Кожна літера завжди замінюється тією самою літерою, тому кількість поява переходить без змін, змінюється лише «назва» літери.
3. **Коли таблиця оборотна?** Коли це бієкція: довжина 33, лише літери алфавіту, кожна рівно один раз.
4. **Додавання і віднімання за модулем у Віженера.** $C = (P + K) \bmod 33$. При розшифруванні $P = (C - K) \bmod 33$, а від'ємний результат зводять до 0..32 додаванням 33.
5. **Чому короткий повторюваний ключ вразливий?** Літери на відстані, кратній l, зашифровані тим самим зсувом. Повтори фрагментів (метод Касіскі) або IC дають l, а потім кожен стовпець розв'язується частотним аналізом як шифр Цезаря.

6. **Як побудувати обернену перестановку?** Для кожного j покласти `inv[π_j] = j`. Для (3, 2, 5, 1, 4) отримуємо (4, 2, 1, 5, 3).
7. **Чому політика неповного блока — частина специфікації?** Від неї залежать шифротекст, його довжина і правило розшифрування. Без неї різні реалізації несумісні, а розшифрування може втратити або додати символи.
8. **Які символи просувають ключ Віженера?** Лише 33 літери алфавіту після нормалізації. Пробіли, цифри й розділові знаки ключ не просувають.
9. **Для чого NFC?** Та сама літера (наприклад, Й чи Й) може бути закодована одним або двома кодовими точками. NFC зводить їх до одного подання, тож пошук в алфавіті та порівняння результатів працюють однаково.
10. **Тестовий вектор чи довільний приклад?** Вектор фіксує вхід, ключ і точний очікуваний вихід, узгоджені специфікацією. Він виявляє розбіжності між реалізаціями, тоді як довільний приклад перевіряє лише узгодженість реалізації самої з собою.
11. **Чому D(E(x)) = x недостатньо для безпеки?** Цю властивість має навіть тотожне перетворення. Безпека вимагає, щоб шифротекст не розкривав інформації без ключа, а класичні шифри зберігають частоти та структуру.
12. **Що дає автентифіковане шифрування крім конфіденційності?** Цілісність і автентичність: будь-яка зміна шифротексту, nonce чи пов’язаних даних виявляється, і розшифрування відмовляє.